IT草根

Lucene 6.0 实战（1）–创建索引

2016-05-20 | 2017-11-20 | Programming Notes | 2450

引言

Lucene6.0于**2016年4月8日**发布，要求最低Java版本是**Java 8**。

相信大多数公司的数据库都需要采用分库分表等一些策略，而对于某些特定的业务需求，分别从不同的库不同的表中去检索特定的数据显得比较繁琐，而Lucene正好可以解决某些特殊需求，对于不同库不同表中的数据先建立全量索引，然后将需要检索的数据写入某个单独的表中，供其它业务需求方查询，以后的每天只需要做增量索引并写入数据表即可。

鉴于最近一直在做Lucene相关方面的工作，而本人一向又比较喜欢使用最新发布的版本，而网络上这类资源极少，故将一些要点及示例整理出来，本文主要从实战角度来介绍Lucene 6.0的使用，不涉及过多原理方面的东西，但是对于一些核心点也会有所提及。

Lucene为什么这么流行

Lucene是一个高效的，基于Java的全文检索库，生活中数据主要分为两种：结构化数据和非结构化数据。一般使用的XML、JSON、数据库等都是结构化数据，非结构化数据也叫全文数据，而这种全文数据正是Lucene的用武之地。全文检索主要有两个过程，索引创建（Indexing）和搜索索引（Search）。

Lucene是很多搜索引擎的一个基础实现，被很多大公司所采用，例如Netflix，MySpace，LinkedIn，Twitter，IBM等。可以通过如下几点特性对Lucene有个大概的认识

- 在现代的硬件上一小时可以索引150GB的数据
- 索引20GB的文本文件，产生的索引文件大概是4-6GB
- 只需要1MB的堆内存
- 可定制的排序模型
- 支持多种查询类型
- 通过特定的字段搜索
- 通过特定的字段排序
- 近实时的索引和搜索
- Faceting，Grouping，Highlighting，Suggestions等

鑒于Lucene这么多强大的特性以及流行度，有很多种基于Lucene的搜索技术，其中最流行的两个是**Apache Solr**和**Elastic search**，当然还有许多其它不同语言的Lucene实现：

猜你喜欢

- **CLucene**: Lucene implementation in C++ (http://sourceforge.net/projects/clucene/)
- **Lucene.Net**: Lucene implementation in Microsoft.NET (http://incubator.apache.org/lucene.net/)
- **Lucene4c**: Lucene implementation in C (http://incubator.apache.org/lucene4c/)
- **LuceneKit**: Lucene implementation in Objective-C, Cocoa/GNUstep support (https://github.com/tcurdt/lucenekit)
- **Lupy**: Lucene implementation in Python (RETIRED) (http://www.divmod.org/projects/lupy)
- **NLucene**: This is another Lucene implementation in .NET (out of date) (http://sourceforge.net/projects/nlucene/)
- **Zend Search**: Lucene implementation in the Zend Framework for PHP 5 (http://framework.zend.com/manual/en/zend.search.html)
- **Plucene**: Lucene implementation in Perl (http://search.cpan.org/search?query=plucene&mode=all)
- **KinoSearch**: This is a new Lucene implementation in Perl (http://www.rectangular.com/kinosearch/)
- **PyLucene**: This is GCJ-compiled version of Java Lucene integrated with Python (http://pylucene.osafoundation.org/)

- **MUTIS**: Lucene implementation in Delphi (http://mutis.sourceforge.net/)
- **Ferret**: Lucene implementation in Ruby (http://ferret.davebalmain.com/trac/)
- **Montezuma**: Lucene implementation in Common Lisp (http://www.cliki.net/Montezuma)

存储索引

索引由Lucene按照特定的格式创建，而创建出来的索引必然要存储在文件系统之上，Lucene在文件系统中存储索引的最基本的抽象实现类是BaseDirectory，该类继承自Directory，BaseDirectory有两个主要的实现类：

- FSDirectory：在文件系统上存储索引文件，有六个子类，如下是三个常用的子类
  - SimpleFSDirectory：使用Files.newByteChannel实现，对并发支持不好，它会在多线程读取同一份文件时进行同步操作
  - NIOFSDirectory：使用Java NIO中的FileChannel去读取同一份文件，可以避免同步操作，但是由于Windows平台上存在Sun JRE bug，所以在Windows平台上不推荐使用
  - MMapDirectory：在读取的时候使用内存映射IO，如果你的虚拟内存足够容纳索引文件大小的话，这是一个很棒的选择
- RAMDirectory：在内存中暂存索引文件，只对小索引好，大索引会出现频繁GC

通常情况下，如果索引文件存储在文件系统之上，我们无需自己选择使用FSDirectory的某个实现子类，只要使用FSDirectory中的open(Path path)方法即可，英文doc如下：

> Creates an FSDirectory instance, trying to pick the best implementation given the current environment. The directory returned uses the NativeFSLockFactory. The directory is created at the named location if it does not yet exist.

该方法可以自动根据当前使用的系统环境而选择一个最佳的实现子类，其选择策略是

- 对于Linux，MacOSX，Solaris，Windows 64-bit JREs返回MMapDirectory
- 对于其它非Windows上的JREs，返回NIOFSDirectory
- 对于其它Windows上的JREs，返回SimpleFSDirectory

MMapDirectory就目前来说，是比较好的实现。它使用virtual memory和mmap来访问磁盘文件。一般的方法都是依赖系统调用在文件系统cache以及Java heap之间拷贝数据。那么怎么才能直接访问文件系统cache呢？这就是mmap的作用！

简单说MMapDirectory就是把lucene的索引当作swap file来处理。mmap()系统调用让OS把整个索引文件映射到虚拟地址空间，这样Lucene就会觉得索引在内存中。然后Lucene就可以像访问一个超大的byte[]数据（在Java中这个数据被封装在ByteBuffer接口里）一样访问磁盘上的索引文件。Lucene在访问虚拟空间中的索引时，不需要任何的系统调用，CPU里的MMU（memory management unit）和TLB（translation lookaside buffers, 它cache了频繁被访问的page）会处理所有的映射工作。如果数据还在磁盘上，那么MMU会发起一个中断，OS将会把数据加载进文件系统Cache。如果数据已经在cache里了，MMU/TLB会直接把数据映射到内存，这只需要访问内存，速度很快。程序员不需要关心paging in/out，所有的这些都交给OS。而且，这种情况下没有并发的干扰，唯一的问题就是Java的ByteBuffer封装后的byte[]稍微慢一些，但是Java里要想用mmap就只能用这个接口。还有一个很大的优点就是所有的内存issue都由OS来负责，这样没有GC的问题。

索引核心类

执行简单的索引过程需要用到以下几个类：

- **IndexWriter**：负责创建索引或打开已有索引
- **IndexWriterConfig**：持有创建IndexWriter的所有配置项
- **Directory**：描述了Lucene索引的存放位置，它的子类负责具体指定索引的存储路径
- **Analyzer**：负责文本分析，从被索引文本文件中提取出语汇单元。对于文本分析器Analyzer，需要注意一点，就是使用哪种Analyzer进行索引创建，查询的时候也要使用哪种Analyzer查询，否则查询结果不正确。
- **Document**：代表一些域（Field）的集合，你可以将Document对象理解为虚拟文档-例如Web页面、E-mail信息或者文本文件
- **Field**：索引中的每个文档都包含一个或多个不同命名的域，每个域都有一个域名和对应的域值
- **FieldType**：描述了Field的各种属性，在不使用某种具体的Field类型（例如StringField，TextField）时需要用到此类

创建索引

索引的创建方式有三种，通过IndexWriterConfig.OpenMode进行指定，分别是

- CREATE：创建一个新的索引或者覆写已经存在的索引
- APPEND：打开一个已经存在的索引
- CREATE_OR_APPEND：如果不存在则创建新的索引，如果存在则追加索引

```
/**
 * 创建索引写入器
 *
 * @param indexPath
 * @param create
 * @throws IOException
 */
public IndexWriter getIndexWriter(String indexPath, boolean create) throws IOException {
    IndexWriterConfig indexWriterConfig = new IndexWriterConfig(new StandardAnalyzer());
    if (create) {
        indexWriterConfig.setOpenMode(IndexWriterConfig.OpenMode.CREATE);
    } else {
        indexWriterConfig.setOpenMode(IndexWriterConfig.OpenMode.CREATE_OR_APPEND);
    }
    Directory directory = FSDirectory.open(Paths.get(indexPath));
    IndexWriter indexWriter = new IndexWriter(directory, indexWriterConfig);
    return indexWriter;
}
```

如果仅仅做测试用，还可以将索引文件存储在内存之中，此时需要使用RAMDirectory

```
public class LuceneDemo {
    private Directory directory;
    private String[] ids = {"1", "2"};
    private String[] unIndex = {"Netherlands", "Italy"};
    private String[] unStored = {"Amsterdam has lots of bridges", "Venice has lots of canals"};
    private String[] text = {"Amsterdam", "Venice"};
    private IndexWriter indexWriter;

    private IndexWriterConfig indexWriterConfig = new IndexWriterConfig(new StandardAnalyzer());

    @Test
    public void createIndex() throws IOException {
        directory = new RAMDirectory();
        //指定将索引创建信息打印到控制台
        indexWriterConfig.setInfoStream(System.out);
        indexWriter = new IndexWriter(directory, indexWriterConfig);
        indexWriterConfig = (IndexWriterConfig) indexWriter.getConfig();
        FieldType fieldType = new FieldType();
        fieldType.setIndexOptions(IndexOptions.DOCS_AND_FREQS_AND_POSITIONS);
        fieldType.setStored(true);//存储
        fieldType.setTokenized(true);//分词
        for (int i = 0; i < ids.length; i++) {
            Document document = new Document();
            document.add(new Field("id", ids[i], fieldType));
            document.add(new Field("country", unIndex[i], fieldType));
            document.add(new Field("contents", unStored[i], fieldType));
            document.add(new Field("city", text[i], fieldType));
            indexWriter.addDocument(document);
        }
        indexWriter.commit();
    }
}
```

NOTES：在调用IndexWriter的close()方法时会自动调用commit()方法，在调用commit()方法时会自动调用flush()方法。所以一般无需这样操作

```
indexWriter.flush();
```

```
indexWriter.commit();
indexWriter.close();
```

控制台输出索引创建信息如下：

> IFD 0 [2016-05-19T07:10:21.127Z; main]: init: current segments file is “segments”; deletionPolicy=org.apache.lucene.index.KeepOnlyLastCommitDeletionPolicy@691a7f8f
> IFD 0 [2016-05-19T07:10:21.167Z; main]: delete []
> IFD 0 [2016-05-19T07:10:21.167Z; main]: now checkpoint “” [0 segments ; isCommit = false]
> IFD 0 [2016-05-19T07:10:21.167Z; main]: delete []
> IFD 0 [2016-05-19T07:10:21.167Z; main]: 0 msec to checkpoint
> IW 0 [2016-05-19T07:10:21.167Z; main]: init: create=true
> IW 0 [2016-05-19T07:10:21.168Z; main]:
> ...
> ...
> DW 0 [2016-05-19T07:10:21.271Z; main]: main finishFullFlush success=true
> IW 0 [2016-05-19T07:10:21.271Z; main]: startCommit(): start
> IW 0 [2016-05-19T07:10:21.271Z; main]: skip startCommit(): no changes pending
> IFD 0 [2016-05-19T07:10:21.271Z; main]: delete []
> IW 0 [2016-05-19T07:10:21.271Z; main]: commit: pendingCommit == null; skip
> IW 0 [2016-05-19T07:10:21.271Z; main]: commit: took 0.4 msec
> IW 0 [2016-05-19T07:10:21.271Z; main]: commit: done
> IW 0 [2016-05-19T07:10:21.271Z; main]: rollback
> IW 0 [2016-05-19T07:10:21.271Z; main]: all running merges have aborted
> IW 0 [2016-05-19T07:10:21.271Z; main]: rollback: done finish merges
> DW 0 [2016-05-19T07:10:21.271Z; main]: abort
> DW 0 [2016-05-19T07:10:21.271Z; main]: done abort success=true
> IW 0 [2016-05-19T07:10:21.271Z; main]: rollback: infos=_0(6.0.0):c2
> IFD 0 [2016-05-19T07:10:21.271Z; main]: now checkpoint “_0(6.0.0):c2” [1 segments ; isCommit = false]
> IFD 0 [2016-05-19T07:10:21.272Z; main]: delete []
> IFD 0 [2016-05-19T07:10:21.272Z; main]: 0 msec to checkpoint
> IFD 0 [2016-05-19T07:10:21.272Z; main]: delete []
> IFD 0 [2016-05-19T07:10:21.272Z; main]: delete []

删除文档

在IndexWriter中提供了从索引中删除Document的接口，分别是

- deleteDocuments(Query... queries)：删除所有匹配到查询语句的Document
- deleteDocuments(Term... terms)：删除所有包含有terms的Document
- deleteAll()：删除索引中所有的Document

NOTES: deleteDocuments(Term... terms)方法，只接受Term参数，而Term只提供如下四个构造函数

- Term(String fld, BytesRef bytes)
- Term(String fld, BytesRefBuilder bytesBuilder)
- Term(String fld, String text)
- Term(String fld)

所以我们无法使用deleteDocuments(Term... terms)去删除一些非String值的Field，例如IntPoint，LongPoint，FloatPoint，DoublePoint等。这时候就需要借助传递Query实例的方法去删除包含某些特定类型Field的Document。

```
@Test
public void testDelete() throws IOException {
    RAMDirectory ramDirectory = new RAMDirectory();
```

```
    IndexWriter indexWriter = new IndexWriter(ramDirectory, new IndexWriterConfig(new StandardAnalyzer()))
    Document document = new Document();
    document.add(new IntPoint("ID", 1));
    indexWriter.addDocument(document);
    indexWriter.commit();
    //无法删除ID为1的
    indexWriter.deleteDocuments(new Term("ID", "1"));
    indexWriter.commit();
    DirectoryReader open = DirectoryReader.open(ramDirectory);
    IndexSearcher indexSearcher = new IndexSearcher(open);
    Query query = IntPoint.newExactQuery("ID", 1);
    TopDocs search = indexSearcher.search(query, 10);
    //命中，1，说明并未删除
    System.out.println(search.totalHits);

    //使用Query删除
    indexWriter.deleteDocuments(query);
    indexWriter.commit();
    indexSearcher = new IndexSearcher(DirectoryReader.openIfChanged(open));
    search = indexSearcher.search(query, 10);
    //未命中，0，说明已经删除
    System.out.println(search.totalHits);
}
```

参考文献

[1] http://www.cnblogs.com/huangfox/p/3616298.html

坚持原创技术分享，您的支持将鼓励我继续创作！

打赏

本文作者： Wang Xu

本文链接： http://www.codepub.cn/2016/05/20/Lucene-6-0-in-action-1-Index-creation/

版权声明： 本博客所有文章除特别声明外，均采用 CC BY-SA 4.0 许可协议。转载请注明出处！

Lucene

Linux下使用java -jar运行可执行jar包的正确方式

Lucene 6.0 实战（2）-各种Field及排序操作

分享到： 收藏夹 复制网址 邮件 微信 QQ空间 腾讯微博 豆瓣 更多 0